

WAPDA Staff-Colony Maintenance Cost Prediction

What We Did and Why

A plain-language record of the full project journey

Saeed Ahmad · CAIP Batch 8 · August 2026

From data assessment to model comparison and Phase 2 improvements

Purpose of this document

This document is not the short CAIP exam report. It is a longer explanation for anyone who wants to understand the whole project: what problem we chose, what data we had, what we built, which models we tried, and why we made each major decision.

The project belongs to the WAPDA staff-colony maintenance planning idea. The work was done carefully so that results are honest, repeatable, and safe to explain to managers without overstating what the numbers prove.

1. The problem we wanted to solve

WAPDA maintains houses and apartments in staff colonies. Each year, maintenance money must be planned for civil work, plumbing, electrical repairs, roofing, painting, sewerage, and emergency fixes.

Today, budgets often rely on:

- past averages across many houses,
- manual inspections,
- complaint lists,
- engineering judgement.

That works at a rough level, but it treats very different properties as if they were the same. An old bungalow with repeated plumbing problems is not the same as a recently repaired flat, yet both may receive the same average allowance.

Our goal: for one house or apartment, estimate eligible maintenance cost for the **next twelve months**, and also flag properties that are likely to become **high-cost** cases.

Two jobs, not one:

1. **Budget number** — how much will this property cost next year?
2. **Priority list** — which properties deserve early attention?

The original proposal also planned a web application where a user could pick a property and see the estimate. That application is still future work. This project completed the data and modelling foundation.

2. What we found in the WAPDA / WASC files

We started with real WAPDA-related material in the DatasetOfCAIP/ folder:

- occupancy and inventory lists for **101 residential units** (categories A–F),

- property presentations and land/asset documents,
- a repair-and-maintenance ledger with hundreds of transactions.

These files were valuable. They defined the **real WAPDA problem**: 101 in-scope residential units, eligible maintenance types, shared-building cost sharing, and privacy rules.

But they were **not enough** to train a property-level machine learning model because:

- the ledger is mostly **account-level**, not cleanly linked to one house and one work order,
- many charges cannot be tied to a completed job on a specific property,
- names, CNIC, salaries, and free-text remarks must stay private,
- linked WAPDA work-order and invoice extracts were **not released** for training under current privacy rules.

Important rule we kept: a missing ledger line is **not** the same as zero cost. Zero is only valid when coverage proves that no eligible work happened.

Decision: keep WASC files as private **seed evidence** for scope and policy, and build a separate **public training dataset** mapped to the same WAPDA data model.

3. The WAPDA data model we fixed first

Before any modelling, we wrote down exactly what one training row means in WAPDA terms:

Table 1: The WAPDA data model (the rulebook for every row).

Element	Meaning
One row	One residential property on one cutoff date
Target	Eligible maintenance cost for the next 12 months
Included	Routine, corrective, emergency building work + fair share of shared building cost
Excluded	Major renovation, rebuilding, appliances, land, rent, revenue entries
Scope	101 WASC residential units only (not offices, roads, mosque, hospital, etc.)
High-cost flag	Top 20% of costs in the training fold
Features	Only facts known on or before the cutoff date

This model came from WAPDA/WASC framing. The training table was then **filled** using public housing survey data that could be mapped to these fields. That is why we say the dataset is **made on the WAPDA data model**, even though the dollar amounts come from a U.S. survey proxy, not from WAPDA invoices.

4. Phase 1 — Dataset preparation

Phase 1 was about building a clean, honest dataset and making sure the evaluation could not cheat.

4.1 Step 1: Register and validate sources

We registered every source file with hashes and version information so nothing could change silently. Raw WASC files stayed untouched. Public sources (RHFS and AHS) were validated for integrity before use.

Why: reproducibility and trust. If someone asks “where did this number come from?”, we can point to a registered file and a transformation log.

4.2 Step 2: Two public dataset slices (kept separate)

We built two real public slices, but **never mixed their labels**:

Table 2: Public training slices (separate tasks).

Release	What it is	Properties		Labels
RHFS 2024	Cross-sectional U.S. rental properties	4,425	1,488 annual repair labels	
AHS 2015–2023	Longitudinal U.S. housing survey	36,623 linked units		86,125 pairs

RHFS was useful as an early slice and proof of the pipeline. **AHS** became the main modelling corpus because it supports **time-based** learning: we see a house in one survey year and its maintenance cost in the next survey wave (two years later).

The AHS gate passed because we had far more than the 500-property minimum (36,623 units).

Why AHS for the main experiments: maintenance forecasting needs history and a future label on the same house. AHS gives linked before/after pairs. RHFS is a different kind of label and stays a separate experiment view.

4.3 Step 3: Map survey fields onto the WAPDA model

Example mappings:

- building type BLD → WAPDA building type,
- year built YRBUILT → property age,
- rooms and bedrooms → size proxies,
- earlier MAINTAMT → past maintenance cost (feature),
- later MAINTAMT → future maintenance cost (target).

Why mapping matters: the model learns WAPDA-shaped inputs, not random survey columns. When WAPDA operational data becomes available later, the same field names and rules can be reused.

4.4 Step 4: Split the data without leakage

We used a **grouped temporal split**. Each house belongs to only one of training, validation, or test. The same house never teaches the model and also appears in the final exam.

Table 3: AHS split (ahs-grouped-temporal-v1).

Set	Houses	Rows	Used for
Training	10,103	13,871	Learn cleaning rules and fit models
Validation	7,067	15,400	Choose settings (pick lowest MAE)
Test	19,453	56,854	Final score only

We also kept a second test view that **removes 2023 labels**, because the 2023 survey changed its maximum reportable maintenance amount. That sensitivity check stops us from confusing a survey rule change with model skill.

Why this split design: random row splits would let the same house appear in both training and testing. That would inflate scores and mislead decision-makers.

4.5 Step 5: Preprocessing (training data only)

On training rows only, we learned:

- how to fill missing numeric values (medians),
- how to encode categories,
- the high-cost cutoff (USD 1,428 = top 20% of training costs).

Then we applied those rules to validation and test without looking at their labels for learning.

The first feature matrix had **205 columns**. The target was never changed, imputed, or clipped.

Why training-only preprocessing: if validation or test labels influence cleaning rules, the model is graded on data it already saw. That is a common source of hidden cheating in machine learning projects.

5. Phase 1 — First model comparison (Experiment 1)

Once the dataset was ready, we ran a fixed, documented comparison: `ahs-baselines-models-v1`.

5.1 What we compared

Simple baselines (easy to explain to a manager):

- **Training median** — predict the middle cost seen in training,
- **Type median** — predict the middle cost for that building type,
- **Prior cost** — predict last period's maintenance cost.

Learning models (from the proposal):

- Linear regression,
- Random Forest,
- Gradient boosting.

Why include baselines: a complex model must beat a simple, transparent rule. If it cannot, there is no reason to deploy it.

5.2 How we measured success (in plain words)

- **MAE** — average dollar error. Lower is better. Main score for picking models.
- **RMSE** — like MAE, but large mistakes hurt more.
- **High-cost F1** — how well we find expensive houses (above USD 1,428). Higher is better.

5.3 Main Phase 1 results (primary test set)

What this showed:

- Gradient boosting had the **lowest test MAE** among fitted models.
- Type median was still best on the **validation** set (827.95 USD), which is the score we chose models by.
- Prior cost was best at finding **high-cost** houses (F1 = 0.410).
- Improvements were real but small — tens of dollars on errors that are already hundreds of dollars.

Table 4: Experiment 1 — key test-set scores (USD).

System	Test MAE	Test RMSE	High-cost F1
Type median	962.81	2361.65	—
Prior cost	1058.25	2358.81	0.410
Linear regression	950.91	2182.90	0.384
Random Forest	956.54	2188.13	0.397
Gradient boosting	949.15	2188.68	0.356

5.4 The selection policy (why we did not declare a “winning AI model”)

We wrote a rule **before** looking at excuses after the test:

1. Pick by **validation MAE**, not test MAE.
2. Also report the no-2023 sensitivity test and high-cost F1.
3. Promote a fitted model only if it wins validation **and** is at least as good on the sensitivity test.

Under that rule:

- **Official cost estimator:** type median,
- **Official high-cost reference:** prior cost,
- **Promoted fitted model:** none.

Why this matters: it stops us from chasing a small test-set win that may not survive a rule change or a new data year. It also matches public-sector needs: a simple rule people can audit often beats a black box by a few dollars.

6. Experiment 2 — Log1p target transform (tried and rejected)

What we did: train the same models on $\log(1 + cost)$ instead of raw dollars, then convert predictions back to dollars for scoring.

Why we tried it: maintenance costs are skewed (many small, few very large). Log transforms often help tree models with heavy tails.

What happened: average error sometimes looked slightly better on validation, but **high-cost detection collapsed**. On the test set, gradient boosting F1 fell from 0.356 to 0.008.

Decision: do not use log1p training as the reporting path. The problem was not mainly “wrong scale”; feature quality and model settings mattered more.

7. Phase 2 — Model improvement sequence

After Experiment 1 and the log1p check, we opened a structured Phase 2 plan (`Documentation/next_steps.md`). The order was deliberate: **understand features before heavy tuning**.

7.1 Phase 2, Step 1 — Feature audit

What we did: reviewed all 205 model columns on 13,871 training rows.

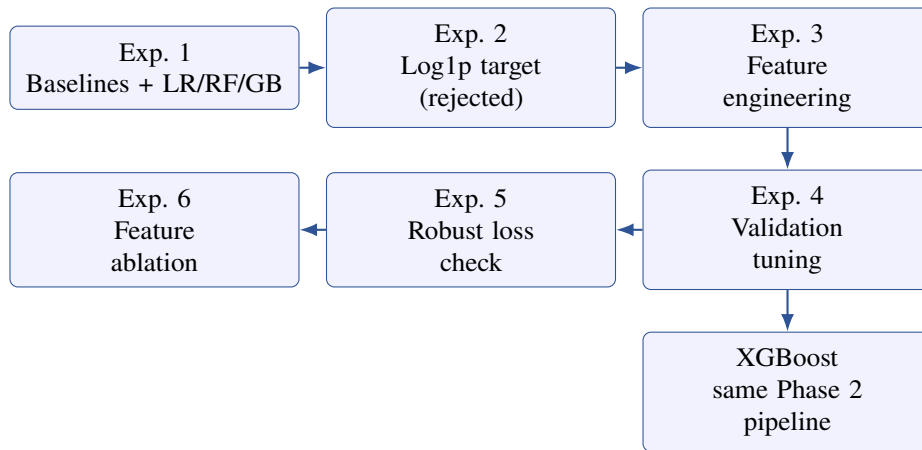


Figure 1: Phase 2 experiment sequence. Feature engineering came before tuning on purpose.

Why: tuning a weak feature set wastes time. We needed to know what was missing, redundant, or unsuitable.

Findings:

- 26 harmonized survey fields expanded to 205 encoded columns,
- roof leak field had very high missingness (kept missing flag, dropped unreliable values),
- no continuous floor-area field (so “cost per sqft” was not directly available),
- survey weight is for survey design, not for WAPDA operations.

7.2 Phase 2, Step 2 — Feature engineering (Experiment 3)

We added five simple derived features that are easier for models to use:

- property age = survey year minus year built,
- log of past maintenance cost,
- past cost divided by number of rooms,
- rooms per bedroom,
- count of condition/defect flags.

The matrix grew from 205 to **215 columns**. Preprocessing audits passed.

Result alone: small change on primary test MAE (gradient boosting 949.15 → 950.42). Engineering by itself was not a magic fix, but it prepared better inputs for tuning.

Why still worth doing: derived features match how engineers think (age, burden, intensity), and they unlock gains when combined with tuning.

7.3 Phase 2, Step 3 — Preprocessing check

We confirmed both preprocessors (original and engineered) learn only from training data. Linear regression uses standardized numeric inputs; tree models use the same encoded matrix.

Why: different models must still follow the same leakage-safe contract.

7.4 Phase 2, Step 4 — Validation-only tuning (Experiment 4)

What we did: search hyperparameters for Random Forest and gradient boosting on the **validation set only**, then refit the best settings on training data. Linear regression settings stayed fixed.

Why validation-only: the test set must remain a true final exam. Tuning on test data would be another form of cheating.

Primary test MAE after tuning:

- Random Forest: 956.54 → **946.57**
- Gradient boosting: 949.15 → **945.27**

Tuning helped tree models by about 4–10 USD on average error — modest but real.

7.5 Phase 2, Step 5 — Robust loss comparison (Experiment 5)

What we did: compare squared error vs absolute error vs Huber loss for gradient boosting.

Why: absolute error focuses on typical misses and can ignore expensive outliers — we needed to see if that trade was acceptable.

Result:

- Absolute loss: test MAE **899.84** (best MAE in this sequence),
- But high-cost F1 fell to **0.065** (almost stops finding expensive houses),
- Squared loss kept better balance for prioritisation.

Lesson: a lower average error can be the wrong goal if the organisation also needs a priority list.

7.6 Phase 2, Step 6 — Inflation sensitivity (deferred)

We documented a plan but did not run it as a main experiment because the 2023 survey cap change already dominates nominal-dollar drift. A separate inflation adjustment would not cleanly separate “price inflation” from “survey rule change”.

7.7 Phase 2, Step 7 — Feature ablation (Experiment 6)

What we did: train gradient boosting with different feature groups removed.

Why: tell WAPDA which information actually drives forecasts.

Key result: removing past maintenance features raised test MAE from 947.79 to **979.67** (about USD 32 worse).

Actionable conclusion: a future WAPDA system needs reliable **property-level maintenance history**. Building characteristics alone are not enough.

7.8 Phase 2 extension — XGBoost pipeline

The proposal mentioned advanced tree models; we added XGBoost on the same engineered features and Phase 2 steps:

- XGBoost with fixed settings,
- validation-only tuning (144 combinations),

- absolute-error objective check (same MAE vs F1 tradeoff as gradient boosting).

Best balanced fitted model on the full test set: tuned XGBoost — MAE **944.34**, RMSE **2182.07**, high-cost F1 **0.391**.

Why XGBoost: strong industry default for tabular data; good check that Phase 2 gains were not specific to one sklearn implementation.

8. Software, tests, and governance (why the work is trustworthy)

We did not only train models in notebooks. The project is a Python package (`caip_maintenance`) with:

- config files for splits, preprocessors, and experiments,
- command-line steps for harmonize, split, preprocess, train, and audit,
- automated tests for leakage, metric math, and artifact reload,
- privacy rules: no PII in tests, logs, or reports; raw WASC files stay local.

Why this extra work: CAIP requires real implementation, not just slides. WAPDA would also need auditable pipelines before any operational use.

9. What we can say confidently today

Table 5: Plain-language summary of outcomes.

Topic	Conclusion
Dataset	Built on the WAPDA data model; filled with mapped AHS public data because WAPDA work orders were unavailable
Best simple budget rule	Type median (validation MAE 827.95 USD)
Best simple priority rule	Prior cost (high-cost F1 0.410)
Best tuned learning model	XGBoost (test MAE 944.34; F1 0.391)
Absolute-error models	Lower MAE but poor at finding expensive houses — not suitable alone for prioritisation
Past maintenance history	Strongly important (ablation + prior-cost results)
WAPDA operational forecast	Not validated yet — needs linked WAPDA costs in PKR
Web application	Planned; not built in this phase

10. What remains for a full WAPDA deployment

1. **Operational WAPDA data** — anonymized property IDs, dated work orders, verified costs, shared-building allocations, and explicit coverage rules.
2. **PKR labels** — the current proxy is in U.S. survey dollars.
3. **Web proof of concept** — property lookup, estimate display, high-cost list, colony summaries.
4. **Pilot on selected colonies** — compare predictions to actual outcomes after data access is approved.
5. **Monitoring** — detect when survey or accounting rule changes break model assumptions (we already saw this risk with the 2023 AHS cap).

11. Timeline in one view

Table 6: Major project stages (simplified timeline).

Stage	Status	What happened
Problem & proposal	Done	WAPDA maintenance cost + prioritisation; CAIP scope
WASC data assessment	Done	101-unit inventory; ledger not property-linked enough
Dataset policy	Done	Public multi-source training path approved
RHFS release	Done	First public slice; pipeline proof
AHS release	Done	Main corpus: 86,125 linked pairs
Split & preprocess	Done	Grouped temporal split; training-only fit
Experiment 1	Done	Baselines + LR/RF/GB; selection policy applied
Experiment 2	Done	Log1p target rejected
Phase 2 steps 1–7	Done	Audit, engineering, tuning, robust loss, ablation
XGBoost Phase 2	Done	Tuned XGBoost best balanced learner
CAIP report & code annex	Done	PDF report, zip annex, test source file
Web POC	Later	Not started
WAPDA validation	Blocked	Waiting on operational extracts

12. Closing note

This project did real work on a real public-sector problem under real privacy limits. It did not pretend that survey dollars are WAPDA invoices. It built the WAPDA-shaped data model, tested simple rules before complex models, wrote the selection rules in advance, and ran a full Phase 2 improvement path when the first experiments showed that target transforms alone were not the answer.

The most useful message for WAPDA is practical: **start collecting property-linked maintenance history now**. The models confirm that past spending and building type carry much of the signal. When operational data arrives, the same workflow — split by time, train safely, compare against simple baselines, report honestly — is ready to use.

Related files: `reports/submission/caip_final_report.pdf` (exam report), `Documentation/AHSNextStepsExecution.md` (Phase 2 log), `Documentation/DatasetPolicy.md` (data strategy).